

Study Plan — 14 Weeks

CS201 · Data Structures and Algorithms · Fall 2026

Dr. Haitham A. El-Ghareeb

Fall 2026

Fourteen teaching weeks inside the faculty's 16–17 week semester (*FCIS Bylaw 2026*, pp. 14–15). The **midterm** falls in the window set by the faculty calendar — the date is announced on the [WhatsApp channel](#), not fixed here.

Every week names the module you implement and the test file that grades it, so this plan and the repository cannot quietly drift apart.

The plan

Wk	Topic	Implement	Graded by
1	Why this course, why Python · ADTs · Know-How	—	Homework 1
2	Complexity — Big-O, and measuring instead of asserting	—	viz/complexity.py demo
3	Arrays · dynamic arrays · amortised cost	dsa/dynamic_array.py	tests/test_dynamic_array.py
4	Linked lists — insert, remove, reverse	dsa/linked_list.py	tests/test_linked_list.py
5	Stacks — balanced brackets, shunting-yard	dsa/stack.py	tests/test_stack_queue.py
6	Queues — the $O(n)$ vs $O(1)$ demonstration	dsa/queue.py	tests/test_stack_queue.py
7	Recursion and decomposition	—	practice set
8	Searching — 7 variants · midterm window	dsa/searching.py	tests/test_searching.py
9	Basic sorting · counting sort	dsa/sorting.py	tests/test_sorting.py

Wk	Topic	Implement	Graded by
10	Advanced sorting — merge, quick, heap	<code>dsa/sorting.py</code>	<code>tests/test_sorting.py</code>
11	Trees and traversals	<i>not yet scaffolded</i>	—
12	Heaps and priority queues	<i>not yet scaffolded</i>	—
13	Hash tables — chaining and open addressing	<code>dsa/hashmap.py</code>	<code>tests/test_hashmap.py</code>
14	Graphs · BFS and DFS · where CS303 takes over	<i>not yet scaffolded</i>	—

Week by week

1 — Why this course, and why Python. Languages are sets of decisions. Paradigms, type systems, memory models. Why Python, and why we reimplement what it already gives us. The ladder from variable to data structure. ADT vs implementation. *Know-how*. → [Lecture 01](#)

2 — Complexity. Big-O, Θ and Ω ; best, average and worst case; space vs time. Then the part most courses skip: `measure()` and `plot_growth()` from `viz/complexity.py`, so you see the crossover point and learn what Big-O throws away.

3 — Arrays and dynamic arrays. Contiguous memory, $O(1)$ indexing, the cost of insertion. You build `DynamicArray` on a raw `ctypes` block — not on a `list`. Doubling on resize, and why that makes append **amortised** $O(1)$. Plot `resize_count` and watch the argument become a picture.

4 — Linked lists. Nodes and references. Why insertion at the head is $O(1)$ and indexing is $O(n)$ — the exact mirror of an array. Reversal in place. This is the week to use the debugger and watch `node.next` move.

5 — Stacks. LIFO. One ADT, two honest implementations. Then the applications that justify it: balanced-bracket checking, and infix→postfix by the shunting-yard algorithm.

6 — Queues. FIFO. `SlowQueue` (`list.pop(0)`, $O(n)$) and `CircularQueue` (ring buffer, $O(1)$) are deliberately paired in `dsa/queue.py` so you can measure the difference rather than be told about it.

7 — Recursion. Base case, recursive case, the call stack, stack depth limits, and converting recursion to iteration. Needed before merge sort, quicksort and every tree traversal that follows.

8 — Searching. (*midterm window*) Linear, binary, binary recursive, `lower_bound`, `upper_bound`, jump, exponential, interpolation. Eight functions, one idea: what a sorted invariant buys you.

9 — Basic sorting. Bubble, selection, insertion — each written twice, once plain and once as a generator yielding snapshots, so `viz/animate.py` can animate *your* code. Then counting sort, which beats the $O(n \log n)$ bound by not comparing.

10 — Advanced sorting. Merge sort (divide and conquer, stable, $O(n)$ extra space), quicksort (in place, and why the pivot strategy decides between $O(n \log n)$ and $O(n^2)$), heap sort.

11 — Trees. Terminology, binary trees, binary search trees. Traversals: in-order, pre-order, post-order, level-order. Why an unbalanced BST degrades to a linked list. Named in the official course content, bylaw p. 138.

12 — Heaps and priority queues. The heap property; the array *is* the tree (`viz.draw.draw_array_as_tree` shows this directly). `sift_up`, `sift_down`, build-heap, and the link back to heap sort.

13 — Hash tables. Hashing, collisions, load factor, resizing. Chaining vs open addressing with tombstones — both are in `dsa/hashmap.py`. Why “ $O(1)$ average” is a promise about *average* and what breaks it.

14 — Graphs, and what comes next. Adjacency matrix vs adjacency list. BFS and DFS. Then an honest handover: the weighted-path, greedy, dynamic-programming and NP-completeness material belongs to **CS303 Analysis and Design of Algorithms** (bylaw p. 144), which you take in Junior Fall.

Two notes on scope

Dynamic programming. `dsa/dynamic_programming.py` is already scaffolded — `fib_naive` / `fib_memo` / `fib_table`, coin change, LCS, edit distance, knapsack, LIS. The bylaw assigns this material to **CS303**, not CS201, so it is **optional enrichment** here, not examinable. Treat it as a bridge. If you want a head start on Junior year, this is where to spend a weekend.

Three modules are missing. The official CS201 content (bylaw p. 138) names **trees**, **heaps**, **priority queues** and **graphs**, and the repository has no `dsa/tree.py`, `dsa/heap.py` or `dsa/graph.py` yet. Weeks 11, 12 and 14 are planned; their skeletons and tests are the next thing to be added. This is recorded here rather than left as a surprise.

Grading, in one line

SA/PE **20** · Midterm **15** · Oral **5** · Final **60** — and you need $\geq 60\%$ overall, $\geq 30\%$ of the final, and $\geq 75\%$ attendance. Full detail and sources: [00-course-guide.md](#).